

Build Plan — Offline-First Review Tracker (PWA)

The complete plan of action, end to end. Your `requirements_v3` rows are the base; a few necessary additions are marked **[added]**. Every phase lists what I hand you, what you do, the git step, and how we know it passed. Each phase also lists the requirement IDs it **Covers**, so you can check coverage against your own file.

How we work (the agreement)

- **Mode:** I write each phase's code; you commit, push, and run it; you report back exactly what you see. You own git. You can switch to "I guide, you build" on any specific piece — just say so for that part.
- **I cannot run the app.** This environment has no browser or network, and a SQLite-WASM PWA only runs in a real browser with on-device storage. So **you are the test harness**. Each phase: I give code → you run it → you paste back what happens (errors verbatim, a screenshot, or "nothing happened"). I fix against that. This loop is the quality mechanism, which is why code comes per phase, not all at once.
- **Pace:** Quality over speed. We don't start a phase until the previous one passes its "Done when."
- **Each phase ends** with a clear "Done when" so you always know whether it passed before moving on.

Git workflow (same every phase)

Recommended branch-per-phase flow (teaches real git, keeps `main` always-working):

```
bash

git checkout main && git pull          # start from latest
git checkout -b phase-N-short-name    # branch for the phase
# ... add the phase's files ...
git add .
git commit -m "Phase N: <what it does>"
git push -u origin phase-N-short-name
# when it passes "Done when":
git checkout main
git merge phase-N-short-name
git push
```

If you'd rather keep it simple, just commit straight to `main` each phase. Either is fine; I'll write the commit message for each phase.

Project layout (where everything lives)

The web app goes in an `app/` subfolder of the `review-tracking` repo, so your Python reference files (`schema.sql`, `cli.py`, `review_db.py`) stay untouched alongside it.

```
review-tracking/
├─ schema.sql          # Python-era reference (kept, not shipped)
├─ cli.py              # reference
├─ review_db.py        # reference — the spec we port from
└─ app/               # the actual product
    ├─ index.html
    ├─ vite.config.js
    ├─ package.json
    ├─ public/         # static assets incl. PWA icons
    └─ src/
        ├─ db/
        │   ├─ worker.js    # SQLite runs here (background thread)
        │   ├─ client.js   # main-thread ↔ worker message bridge
        │   ├─ schema.sql  # SQLite schema + version table
        │   └─ api.js      # the ported review_db functions
        ├─ ui/
        │   ├─ app.js       # screen routing + shell
        │   ├─ table.js     # the data table
        │   ├─ entry-form.js # add/edit entry
        │   ├─ category-form.js
        │   ├─ filters.js
        │   ├─ hide.js
        │   ├─ export.js
        │   ├─ import.js
        │   ├─ data-screen.js # reset / backup / restore [added]
        │   └─ styles.css
        └─ validate.js      # type-constraint checks (shared)
```

What gets added beyond your requirements (kept minimal) [added]

- **Schema version table** — one tiny table recording the schema version, so future schema changes (you'll have them when you add cloud/accounts) can migrate cleanly instead of breaking existing data.
- **On-screen messages** — a phone has no console you can read, so errors and confirmations show in the UI.
- **Data screen** — houses export, import, and "reset to empty"; the reset also satisfies requirement 11.1 (fresh instance, no data).
- **PWA icons** — install needs a few icon sizes; I'll give exact sizes and the manifest entries at Phase 11.

The full toolchain (every tool, role, where it runs)

Tool	Role	Where
Node.js 20 LTS + npm	install deps, run dev server, build	PC only (dev-time); never shipped
Vite	dev server (live reload) + bundler; loads ES modules, the Worker, and WASM correctly	PC (dev); outputs static <code>dist/</code>
<code>@sqlite.org/sqlite-wasm</code>	the SQLite database engine, as WebAssembly	on device, in a Web Worker
OPFS SAH-pool VFS	persists the SQLite file on the device; needs no special cross-origin headers (simpler hosting)	on device
<code>vite-plugin-pwa</code>	generates the service worker + manifest; precaches the app shell incl. the WASM binary so it runs offline	build-time on PC; service worker runs on device
Vanilla JS + HTML + CSS	the UI	on device
Git + GitHub repo <code>review-tracking</code>	version control + deploy source	PC → GitHub
GitHub Pages + GitHub Actions	free static hosting over HTTPS for install/sharing; Actions builds <code>app/</code> and deploys <code>dist/</code>	cloud (delivery only; nothing runs there at app runtime)

Not in the stack: PostgreSQL, `psycopg2`, any Python web framework. `review_db.py` `schema.sql` are kept as reference specs.

Phase 1 — Scaffold

- **Goal:** a running empty app on localhost, pushed to the repo.
- **You do:**
 1. Install Node.js 20 LTS from nodejs.org.
 2. From the `review-tracking` repo root:

```
bash
```

```
npm create vite@latest app -- --template vanilla
cd app
npm install
npm run dev
```

3. Open the printed URL (usually `http://localhost:5173`); confirm the starter page loads and editing a file live-reloads.
 4. Commit + push (see git workflow; branch `phase-1-scaffold`).
- **I provide:** nothing yet — this is stock Vite.
 - **Done when:** the starter page shows on localhost and the scaffold is on GitHub.
 - **Covers:** foundation.

Phase 2 — SQLite-WASM alive and persisting (prove the engine)

- **Goal:** open an on-device database, write a row, refresh the page, read it back — proving data survives with no server.
- **You do first:** `npm install @sqlite.org/sqlite-wasm`
- **I provide:** `src/db/worker.js` (loads SQLite WASM, opens the DB on the OPFS SAH-pool VFS), `src/db/client.js` (promise-based main-thread bridge), and the `vite.config.js` tweak SQLite-WASM needs (excluding it from dependency pre-bundling), plus a 3-line smoke test you run.
- **You do:** paste the files, `npm run dev`, run the smoke test, refresh the browser, tell me whether the test row is still there.
- **Done when:** a row written before a refresh is readable after it.
- **Covers:** the storage backbone for every data requirement; basis for 10.

Phase 3 — Schema (SQLite translation + subcategories + version table)

- **Goal:** the real tables, created on first run.
- **I provide:** `src/db/schema.sql` — your model re-expressed in SQLite (`SERIAL` → `INTEGER PRIMARY KEY`; `TIMESTAMP` → `TEXT DEFAULT CURRENT_TIMESTAMP`; `NUMERIC CHECK` → `REAL CHECK`; `BOOLEAN` → `INTEGER`; `JSONB` → `TEXT`), the Postgres GIN index dropped, `PRAGMA foreign_keys = ON` for cascades, **subcategories** via a self-referencing `parent_id` on `categories`, and the **schema version table** [added]. Plus the worker logic to run this only on a fresh database.
- **You do:** paste, reload, confirm tables exist and a re-open doesn't duplicate them (I'll give the check).
- **Done when:** a fresh DB initializes with all tables/constraints; re-opening is clean.
- **Covers:** 1.1, 7.1.

- **Note:** index strategy (what to index) stays deferred, as you wanted — revisited when we scale.

Phase 4 — Data-access layer (port `review_db.py` to JS)

- **Goal:** one JS function per operation; all SQL contained here.
- **I provide:** `src/db/api.js` porting every `review_db.py` function (`getCategories`, `addCategory`, `addCategoryColumn`, `getCategoryColumns`, `getCategoryEntries`, `getEntry`, `addEntry`, `updateEntry`, `deleteEntry`) plus new ones for subcategories, category hiding, and bulk delete — reproducing the base-column-vs-JSON-key logic of `update_entry` using SQLite's `json_set`, and throwing on unknown columns. Plus a temporary on-screen test panel to exercise each function.
- **You do:** paste, run each test button, report results.
- **Done when:** every function works from the test panel.
- **Covers:** the engine behind 1, 2, 5, 6, 7.

Phase 5 — UI shell (first time you see it)

- **Goal:** a phone-sized frame listing real categories.
- **I provide:** `index.html`, `src/ui/app.js`, `src/ui/styles.css` — mobile-first single column, touch-sized targets, header + content area, category list from `getCategories`, plus the on-screen message area **[added]**.
- **You do:** paste, run at phone width (DevTools device mode), report how it looks.
- **Done when:** real categories render and the layout is right at phone width.
- **Covers:** first slice of 2; 10.4.

Phase 6 — Read features (the table)

- **Goal:** tap a category → scrollable, sortable table.
- **I provide:** `src/ui/table.js` — renders base + category-specific columns, vertical and horizontal scroll within phone width, tap-to-sort headers.
- **You do:** paste, test scroll both directions and sorting; report.
- **Done when:** entries display, scroll both ways, and sort by any column.
- **Covers:** 2.1, 2.2, 2.2.1, 2.2.2, 2.3.

Phase 7 — Write features (add / edit / delete) + type validation

- **Goal:** create, edit, remove entries; enforce column types.
- **I provide:** `src/ui/entry-form.js` (pick category + optional subcategory, then base fields + that category's extra fields → `addEntry`), edit-in-place (`updateEntry`), multi-select delete with confirm (`deleteEntry`), clear-a-cell-without-deleting-the-row, and `src/validate.js`

enforcing each column's declared type before save (number rejects non-numbers, date uses a picker).

- **You do:** paste, exercise add/edit/multi-delete/clear and bad-type rejection; report.
- **Done when:** all of the above work and bad-typed input is refused.
- **Covers:** 1, 1.2, 1.2.1, 1.2.2, 5, 5.1, 6, 6.1, 6.1.1, 6.2.

Phase 8 — Category management

- **Goal:** create categories/subcategories with typed, constrained columns; hide a category without deleting it.
- **I provide:** `src/ui/category-form.js` — name, optional parent (makes it a subcategory), add extra columns with type + value constraints, and a hide/show toggle (data preserved).
- **You do:** paste, create a category + subcategory with typed columns, confirm it drives the add-entry form, test hiding; report.
- **Done when:** new categories/subcategories with typed constrained columns appear and work; hiding preserves data.
- **Covers:** 7, 7.1, 7.2, 7.2.1, 7.3.

Phase 9 — Filter and hide rows

- **Goal:** filter the table; hide rows.
- **I provide:** `src/ui/filters.js` (multi-value single-column, multi-column combined, remove one, clear all) and `src/ui/hide.js` (manual hide of selected rows, hide-by-column-value rules, unhide view) using the `is_hidden` column.
- **You do:** paste, test all filter and hide paths incl. removal/unhide; report.
- **Done when:** multi-value/multi-column filtering and manual/rule-based hiding all work, with removal/unhide.
- **Covers:** 3, 3.1, 3.2, 3.3, 3.4, 4, 4.1, 4.2, 4.3.

Phase 10 — Export / import + data screen

- **Goal:** back up, restore, move data, start fresh.
- **I provide:** `src/ui/export.js` (serialize all data → file download), `src/ui/import.js` (read file, **validate before inserting**, report problems, then insert), and `src/ui/data-screen.js` housing export/import/reset [**added**].
- **You do:** paste, export, reset to empty, re-import the file, confirm data returns; test a malformed file is rejected with no changes; report.
- **Done when:** an export re-imports cleanly into a fresh install; a bad file is rejected and changes nothing.
- **Covers:** 8, 8.1, 9, 9.1, 9.2, 11.1.

Phase 11 — Make it an installable PWA

- **Goal:** installs to the home screen, own icon, full-screen, runs offline.
- **You do first:** supply one square source image ($\geq 512 \times 512$). I'll tell you how to generate the required sizes (192×192, 512×512, a 512 maskable, and a 180×180 apple-touch-icon for iOS), or I'll give a one-command way to produce them.
- **I provide:** `vite-plugin-pwa` config in `vite.config.js` (manifest: name, icons, `display: standalone`, `start_url`) and precaching set so the service worker caches the app shell including the SQLite WASM binary and worker.
- **You do:** `npm install -D vite-plugin-pwa`, paste config + icons, `npm run build` then `npm run preview`, install to home screen, turn off network, confirm it launches and works.
- **Done when:** the installed app launches full-screen from its icon and works in airplane mode.
- **Covers:** 10, 10.1, 10.2, 10.3, 10.4.

Phase 12 — Ship to others (and onto your own phone)

- **Goal:** real phones, both yours and a friend's; confirm cross-platform.
 - **I provide:** a GitHub Actions workflow that builds `app/` and deploys `dist/` to GitHub Pages, plus the Vite `base` setting for the repo subpath. (Alternative hosts: Netlify or Cloudflare Pages — also free, also HTTPS.)
 - **You do:** add the workflow, push, enable Pages; open the HTTPS URL on a phone, install via "Add to Home Screen," run offline; confirm a fresh install is empty; test on Android, iOS (Safari's manual Add-to-Home-Screen), and desktop. Report per platform.
 - **Done when:** you and a friend each install an independent, empty copy on different platforms and use it offline.
 - **Covers:** 10.5, 11, 11.1, 11.2, 11.2.1.
-

Deferred / out of scope (on purpose)

- **Index strategy** (what to index in SQLite) — revisited when we scale.
- **Accounts + cloud sync / Google sign-in** — your explicit *later* goal. The local-first design sets this up well: all data access goes through `api.js`, and every row has an id + `created_at`/`updated_at`, which is what a future sync layer needs. It slots in behind that layer later; we build none of it now.
- **App Store / Play Store** — excluded by your recorded decision; install is by link.
- **Desktop-specific layout** — desktop works as secondary (10.5); no dedicated design.

Honest risks

- **iOS is the fussy platform.** OPFS needs a recent Safari, and iOS can evict an installed app's storage under storage pressure or long disuse — which is exactly why export/import (Phase

10) is core, not a nicety: it's your recovery path.

- **The table (Phases 6 + 9) is the hardest part.** Sort + multi-column multi-value filter + hide is a lot of UI state in vanilla JS. If it gets unwieldy, the fork is a small UI framework *for that layer only* — the data layer stays untouched.
- **One-time install needs connectivity + HTTPS.** Covered by free static hosting; just not installable onto a brand-new phone with literally no network.
- **I write code blind to your machine.** Mitigated only by the tight per-phase test loop above — which is why we go one phase at a time.